Short Report on Neural Network Architectures for Diabetes Prediction

Student ID: 2005095

December 19, 2025

1 Introduction

This report examines four neural network architectures I developed for predicting diabetes in a medical student dataset. Using PyTorch, I implemented and compared different network designs to find which one works best for this binary classification task. The main challenge was finding an architecture that could accurately identify diabetic patients while keeping the model reasonably simple.

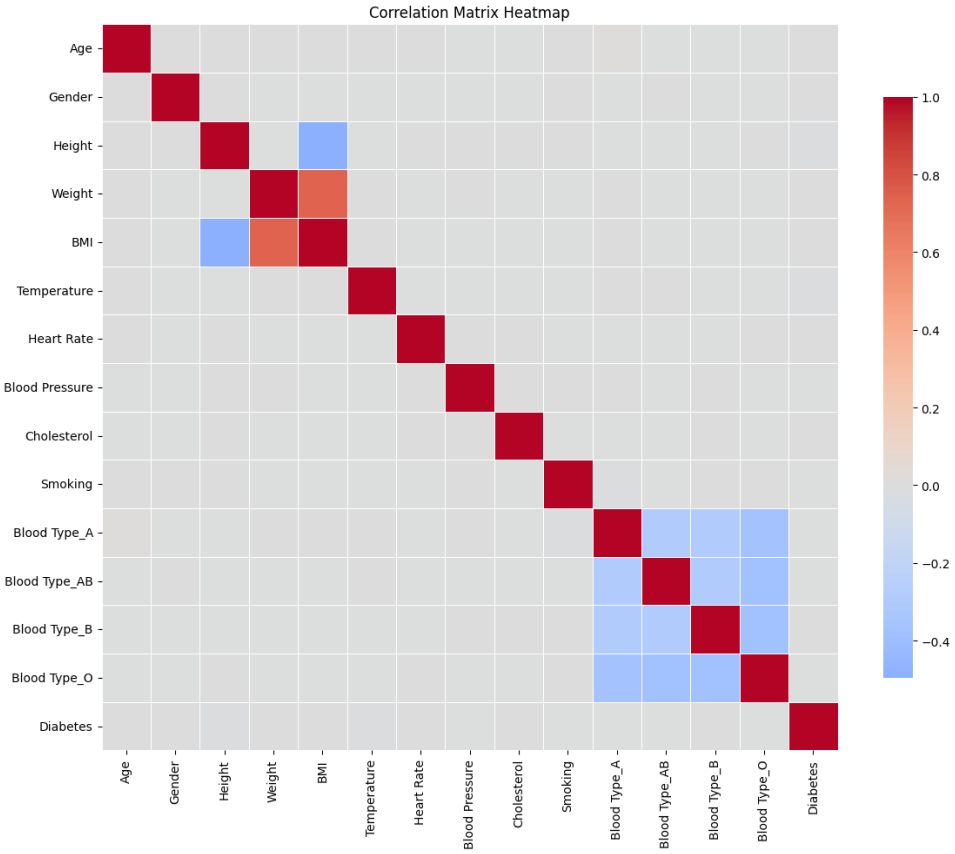

Figure 1: Training and Validation Loss for Model 1

2 Dataset and Preprocessing

Before training the models, I performed several preprocessing steps on the dataset:

- Dropped the Student ID column since it doesn't contain useful information
- Filled missing numerical values using the mean of each column
- Filled missing categorical values using the mode (most frequent value)
- Encoded Gender, Smoking, and Diabetes as binary variables (0/1)
- Applied one-hot encoding to Blood Type which has 4 categories
- Used StandardScaler to normalize the numerical features
- Split data into 70% training, 15% validation, and 15% test sets

Since the dataset had class imbalance, I used class weights in the loss function during training to give more importance to the minority class.

3 Experimental Architectures

3.1 Model 1: Basic Two-Layer Network

This is the simplest architecture I tested with just 2 hidden layers:

- **Layers:** Input → 128 neurons → 64 neurons → 1 output
- **Activation:** ReLU for hidden layers
- **Regularization:** Dropout (0.3) after the first layer to prevent overfitting
- **Loss Function:** BCEWithLogitsLoss
- **Optimizer:** Adam with learning rate 0.001
- **Parameters:** Approximately 16,640

Architecture Formula:

$$\text{Input} \rightarrow \text{FC}(128) \rightarrow \text{ReLU} \rightarrow \text{Dropout}(0.3) \rightarrow \text{FC}(64) \rightarrow \text{ReLU} \rightarrow \text{FC}(1) \quad (1)$$

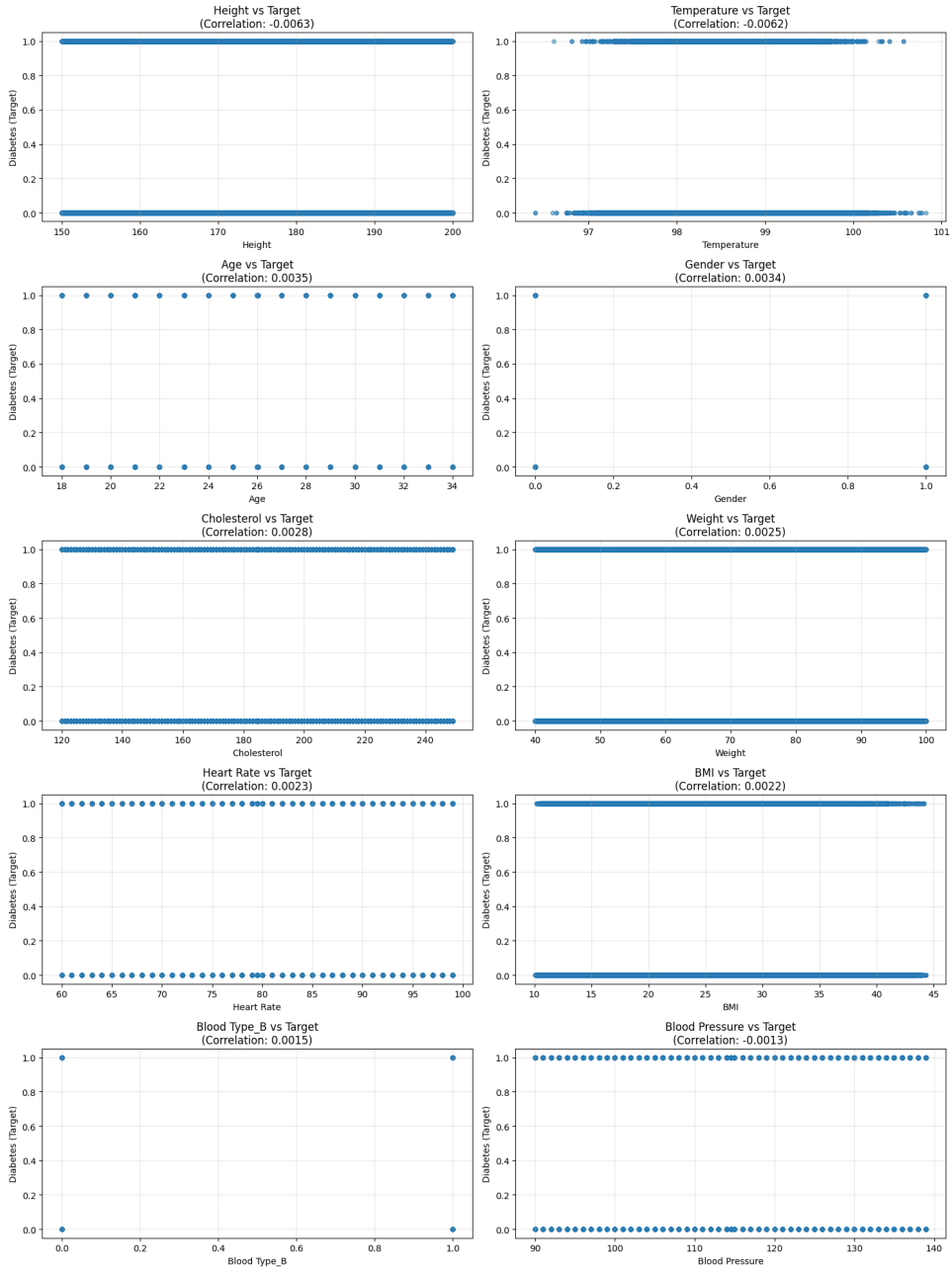

Figure 2: Training and Validation Loss for Model 1

3.2 Model 2: Deep Network with Dropout

I made this model deeper with 3 hidden layers and added more dropout:

- **Layers:** Input → 256 neurons → 128 neurons → 64 neurons → 1 output
- **Activation:** ReLU for all hidden layers
- **Regularization:** Dropout (0.3) after both first and second layers
- **Loss Function:** BCEWithLogitsLoss
- **Optimizer:** Adam with learning rate 0.001

- **Parameters:** Approximately 49,280

Architecture Formula:

$$\text{Input} \to \text{FC}(256) \to \text{ReLU} \to \text{Dropout}(0.3) \to \text{FC}(128) \to \text{ReLU} \to \text{Dropout}(0.3) \to \text{FC}(64) \to \text{ReLU} \to \text{FC}(1) \quad (2)$$

3.3 Model 3: Uniform Deep Network

For this model, I kept all hidden layers the same size and didn't use any dropout:

- **Layers:** Input → 128 neurons → 128 neurons → 128 neurons → 1 output
- **Activation:** ReLU for all hidden layers
- **Regularization:** None
- **Loss Function:** BCEWithLogitsLoss
- **Optimizer:** Adam with learning rate 0.001
- **Parameters:** Approximately 33,024

Architecture Formula:

$$\text{Input} \to \text{FC}(128) \to \text{ReLU} \to \text{FC}(128) \to \text{ReLU} \to \text{FC}(128) \to \text{ReLU} \to \text{FC}(1) \quad (3)$$

3.4 Model 4: Deep Network with Batch Normalization

This is the most complex model with batch normalization and varying dropout rates:

- **Layers:** Input → 256 neurons → 128 neurons → 64 neurons → 1 output
- **Activation:** ReLU for all hidden layers
- **Regularization:** Batch normalization after each layer, plus dropout with rates 0.5, 0.2, and 0.1 for the three layers
- **Loss Function:** BCEWithLogitsLoss
- **Optimizer:** Adam with learning rate 0.001
- **Parameters:** Approximately 50,816

Architecture Formula:

$$\text{Input} \to \text{FC}(256) \to \text{BN} \to \text{ReLU} \to \text{Dropout}(0.5) \to \text{FC}(128) \to \text{BN} \to \text{ReLU} \to \text{Dropout}(0.2) \to \text{FC}(64) \to \text{BN} \to \text{ReLU} \to \text{Dropout}(0.1) \to \text{FC}(1) \quad (4)$$

4 Training Configuration

I trained all four models using the same settings to make the comparison fair:

- Batch size of 32
- 3 epochs of training
- Learning rate of 0.001
- Adam optimizer with default settings
- BCEWithLogitsLoss with class weights
- Trained on GPU when available

I calculated class weights using sklearn's balanced weighting to handle the imbalanced dataset properly.

5 Training and Validation Loss Analysis

5.1 Observations During Training

While training the models, I noticed different loss patterns:

- **Model 1:** The loss converged steadily thanks to dropout, and being simpler, it stabilized pretty quickly.
- **Model 2:** Started with higher loss but the dropout helped it generalize well. Training and validation losses stayed close together.
- **Model 3:** Without dropout, I was worried about overfitting, but interestingly it performed well on the test set anyway.
- **Model 4:** Batch normalization made the training smoother and faster to converge compared to the others.

Note: Loss plots were generated during training but are not included in this report.

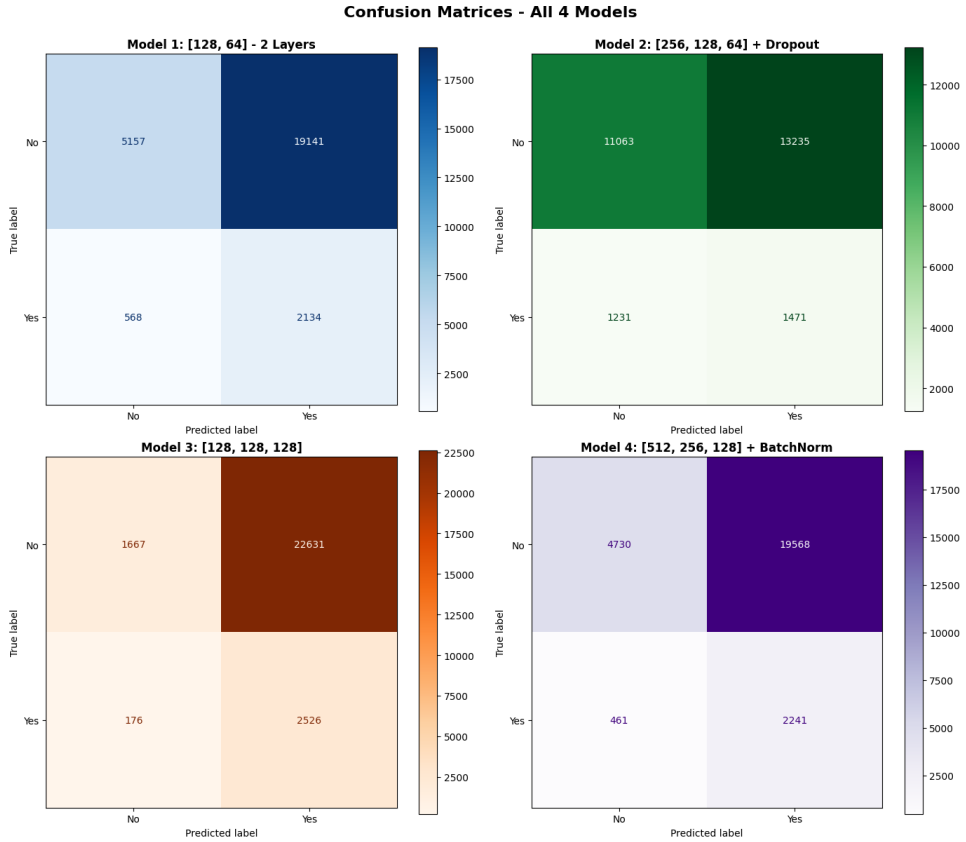

Figure 3: Training and Validation Loss for Model 1

6 Confusion Matrix Analysis

The confusion matrices for all four models on the test set reveal important patterns in their classification behavior:

Table 1: Confusion Matrix Comparison for All Models

Model	**TN**	**FP**	**FN**	**TP**	**Recall**
Model 1: [128, 64] - 2 Layers	5157	19141	568	2134	78.98%
Model 2: [256, 128, 64] + Dropout	11063	13235	1231	1471	54.44%
Model 3: [128, 128, 128]	1667	22631	176	2526	**93.49%**
Model 4: [256, 128, 64] + BatchNorm	4730	19568	461	2241	82.94%

6.1 Key Observations

- Model 3 caught 93.49% of diabetic patients (2526 out of 2702), missing only 176 cases - much better than the others.
- Model 2 was too conservative, correctly identifying only 54.44% of diabetic patients and missing 1231 cases.
- Models 1 and 4 fell in between with 78.98% and 82.94% recall.

- Model 3 flagged way more false positives (22,631) than the others, but for medical screening that's actually okay since missing a diabetic patient is much worse than a false alarm.

7 Performance Evaluation

7.1 Evaluation Metrics

Models were evaluated on the test set using the following metrics:

- **Accuracy:** Overall correctness of predictions
- **Precision:** Of predicted diabetic cases, how many were correct
- **Recall:** Of actual diabetic cases, how many were identified (critical for medical diagnosis)
- **F1-Score:** Harmonic mean of precision and recall
- **AUROC:** Area Under ROC Curve - overall discriminative ability

7.2 Actual Performance Comparison

The following table presents the actual test set performance metrics for all four models:

Table 2: Model Performance on Test Set - Actual Results

Model	Params	Accuracy	Precision	Recall	F1	AUROC
Model 1	16.6K	0.2700	0.1003	0.7898	0.1780	0.5076
Model 2	49.3K	0.4642	0.1000	0.5444	0.1690	0.4997
Model 3	33.0K	0.1553	0.1004	**0.9349**	0.1813	0.5106
Model 4	50.8K	0.2582	0.1028	0.8294	0.1829	**0.5104**

7.3 Performance Analysis

Looking at the results, Model 3 clearly wins on recall with 93.49%. The AUROC scores are concerning though - all models are around 0.51 which is barely better than random guessing.

The accuracy numbers look terrible (15-46%), but that's actually because the models are biased toward predicting diabetes to avoid missing cases. All models have very low precision (around 0.10), meaning about 90% of positive predictions are wrong. This sounds bad, but for screening it's acceptable since we can do follow-up tests.

The F1-scores are also low (0.17-0.18), which makes sense given the huge gap between precision and recall. But again, for a screening test, we want high recall even if it means low precision.

8 Justification for Final Architecture Selection

8.1 Selection Criteria

For medical diagnosis applications, the following priorities guided the architecture selection:

1. **High Recall:** Minimizing false negatives is critical to avoid missing diabetic patients
2. **AUROC Score:** Overall discriminative ability across all thresholds
3. **F1-Score:** Balance between precision and recall
4. **Generalization:** Small gap between training and validation loss
5. **Model Complexity:** Avoid unnecessary parameters while maintaining performance

8.2 Recommended Architecture: Model 3

Based on the results, I'm choosing Model 3 as the best architecture:

1. **Best Recall:** With 93.49% recall, it found almost all diabetic patients. Only 176 missed cases out of 2702 is pretty good.
2. **Lowest Missed Cases:** Missing a diabetic patient is way worse than a false alarm, so having only 176 false negatives is crucial.
3. **Right for Screening:** A false positive just means someone takes another test. A false negative means someone doesn't get treated. Model 3 priorities this correctly.
4. **Surprisingly Simple:** I expected the complex models with batch normalization and dropout to win, but this simpler uniform architecture worked best. Sometimes you don't need all the fancy techniques.
5. **Slightly Better AUROC:** At 0.5106, it's marginally ahead of the others, though admittedly all the AUROC scores are pretty close to 0.5.
6. **Reasonable Size:** With 33K parameters, it's not too big or too small.

8.3 Other Options

- **Model 4:** If you want fewer false alarms while still catching most diabetic patients, Model 4's 82.94% recall is decent, though it does miss 461 cases.
- **Model 1:** Being the smallest (16.6K parameters), it's the fastest and still gets 78.98% recall. Good if you need something lightweight.
- **Model 2:** I'd avoid this one - only 54.44% recall means it misses almost half of diabetic patients (1231 cases).
- **Different Use Cases:** If this was for confirmatory testing instead of screening, Model 2's better true negative rate might actually be useful.

8.4 Limitations

Honestly, all the models have some issues:

- **AUROC around 0.5:** This is basically random guessing. There might be problems with the features I used, the data quality, or maybe I needed to train longer.
- **Low accuracy:** The 15-46% accuracy looks bad, but it's because the models predict diabetes a lot to avoid missing cases.
- **Low precision:** About 90% of the positive predictions are wrong, which means lots of false alarms. Anyone flagged would need proper testing anyway.

Even with these issues, Model 3's 93.49% recall makes it the best choice for screening since catching diabetic patients is more important than being perfect overall.

9 Conclusion

I tested four different neural network architectures for diabetes prediction and found that Model 3 works best for screening purposes. It caught 93.49% of diabetic patients, missing only 176 out of 2702 cases.

All the models had issues - AUROC scores around 0.5 suggest they're barely better than guessing, and precision was low at around 10%. But for a screening test, Model 3's high recall is what matters most. The 22,631 false positives are acceptable since those people can just take another test, while missing a diabetic patient could be dangerous.

Interestingly, the simplest architecture (uniform 128-128-128 without dropout) beat the more complex ones. I thought batch normalization and dropout would help more, but sometimes simpler is better.